# Guía de solución - Caso 1: Cálculo de tarifas de alquiler de vehículos

Elaborado por: Henry Antonio Mendoza Puerta

## Enunciado

El caso solicita calcular el precio del alquiler de un vehículo según el tipo de cliente (regular, corporativo o premium) aplicando el patrón Strategy:

- Interfaz de estrategia con el método calculatePrice(int days, double basePrice).
- Estrategias concretas con tarifas distintas.
- Contexto (VehicleRental) que usa la estrategia seleccionada para calcular el total.
- El sistema asigna la estrategia según el tipo de cliente.

## Cómo aplicamos el patrón Strategy

Cada tarifa (regular, corporativo y premium) es una clase que implementa la interfaz Alquiler.

El contexto AlquilerVehiculo recibe una estrategia y calcula el total sin saber qué tarifa es.

ServicioAlquiler elige la estrategia según el tipo de cliente.

Si aparece un nuevo tipo de cliente, solo se crea una clase nueva y se registra en el servicio: el resto del sistema no cambia.

En esta solución: RentalStrategy = Alquiler, VehicleRental = AlquilerVehiculo, calculatePrice = calcularPrecio.

## Paquetes del proyecto

| Paquete | Contenido |
|---|---|
| model | Clases de datos y enumerados: TipoCliente. |
| strategy | El patrón: interfaz Alquiler, las tres tarifas y el contexto AlquilerVehiculo. |
| service | Elige la estrategia según el tipo de cliente y calcula el total (ServicioAlquiler). |
| view | La ventana (JFrame) en VistaAlquiler. |
| controller | ControladorAlquiler: conecta la ventana con el servicio. |
| (raíz) | Principal: inicia el programa. |

# Paso a paso

## Paso 1. Modelo (TipoCliente)

Enumerado con los tipos de cliente.

Archivo: TipoCliente.java

```
package com.usil.model;

public enum TipoCliente {
    REGULAR, CORPORATIVO, PREMIUM
}
```

## Paso 2. Interfaz de la estrategia (Alquiler)

Método que implementa cada tarifa.

Archivo: Alquiler.java

```
package com.usil.strategy;

public interface Alquiler {

    double calcularPrecio(int dias, double precioBase);
}
```

## Paso 3. Estrategias concretas

Regular: precio completo. Corporativo: 15% de descuento. Premium: 25% de descuento.

Archivo: AlquilerRegular.java

```
package com.usil.strategy;

/** Cliente regular: tarifa completa por cada día. */
public class AlquilerRegular implements Alquiler {

    @Override
    public double calcularPrecio(int dias, double precioBase) {
        return dias * precioBase;
    }
}
```

Archivo: AlquilerCorporativo.java

```
package com.usil.strategy;

/** Cliente corporativo: 15% de descuento. */
public class AlquilerCorporativo implements Alquiler {

    @Override
    public double calcularPrecio(int dias, double precioBase) {
        return dias * precioBase * 0.85;
    }
}
```

Archivo: AlquilerPremium.java

```
package com.usil.strategy;
```

```java
/** Cliente premium: 25% de descuento. */
public class AlquilerPremium implements Alquiler {

    @Override
    public double calcularPrecio(int dias, double precioBase) {
        return dias * precioBase * 0.75;
    }
}
```

## Paso 4. Contexto (AlquilerVehiculo)

Usa la estrategia elegida para calcular el total.

Archivo: AlquilerVehiculo.java

```java
package com.usil.strategy;

/** Contexto: delega el cálculo del precio en la estrategia seleccionada. */
public class AlquilerVehiculo {

    private Alquiler estrategia;

    public void setEstrategia(Alquiler estrategia) {
        this.estrategia = estrategia;
    }

    public double calcularTotal(int dias, double precioBase) {
        return estrategia.calcularPrecio(dias, precioBase);
    }
}
```

## Paso 5. Servicio (ServicioAlquiler)

Elige la estrategia según el tipo de cliente.

Archivo: ServicioAlquiler.java

```
package com.usil.service;

import com.usil.model.TipoCliente;
import com.usil.strategy.Alquiler;
import com.usil.strategy.AlquilerCorporativo;
import com.usil.strategy.AlquilerPremium;
import com.usil.strategy.AlquilerRegular;
import com.usil.strategy.AlquilerVehiculo;

public class ServicioAlquiler {

    private AlquilerVehiculo alquilerVehiculo = new AlquilerVehiculo();

    public double calcular(TipoCliente tipoCliente, int dias, double precioBase) {
        Alquiler estrategia = switch (tipoCliente) {
            case REGULAR -> new AlquilerRegular();
            case CORPORATIVO -> new AlquilerCorporativo();
            case PREMIUM -> new AlquilerPremium();
        };
        alquilerVehiculo.setEstrategia(estrategia);
        return alquilerVehiculo.calcularTotal(dias, precioBase);
    }
}
```

## Paso 6. Crear la vista (VistaAlquiler)

Clic derecho sobre view, New, JFrame Form, nombre VistaAlquiler. En Diseño: JFrame con Border Layout, un JPanel arriba (North) y un JLabel al centro (Center). Agregar al panel estos componentes y renombrarlos (clic derecho, Change Variable Name):

| Nombre | Componente | Texto |
|---|---|---|
| cmbCliente | JComboBox | (vacío) |
| txtDias | JTextField | (vacío) |
| txtPrecioBase | JTextField | (vacío) |
| btnCalcular | JButton | Calcular |
| lblResultado | JLabel | (vacío) |

En Código, fuera del bloque Generated Code, agregar mostrarError y los getters. Archivo completo:

Archivo: VistaAlquiler.java

```
package com.usil.view;

import javax.swing.JOptionPane;

public class VistaAlquiler extends javax.swing.JFrame {

    public VistaAlquiler() {
        initComponents();
        setSize(950, 160);
    }

    public void mostrarError(String message) {
        JOptionPane.showMessageDialog(this, message, "Datos no válidos",
JOptionPane.WARNING_MESSAGE);
    }

    public javax.swing.JComboBox<String> getCmbCliente() {
        return cmbCliente;
    }

    public javax.swing.JTextField getTxtDias() {
        return txtDias;
    }

    public javax.swing.JTextField getTxtPrecioBase() {
        return txtPrecioBase;
    }

    public javax.swing.JButton getBtnCalcular() {
        return btnCalcular;
    }

    public javax.swing.JLabel getLblResultado() {
        return lblResultado;
    }

    /** Inicializa los componentes del formulario. No modificar: lo regenera el editor de
NetBeans (pestaña Diseño). */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated
Code">//GEN-BEGIN:initComponents
    private void initComponents() {

        pnlFormulario = new javax.swing.JPanel();
        lblCliente = new javax.swing.JLabel();
        cmbCliente = new javax.swing.JComboBox<>();
        lblDias = new javax.swing.JLabel();
        txtDias = new javax.swing.JTextField();
        lblPrecioBase = new javax.swing.JLabel();
        txtPrecioBase = new javax.swing.JTextField();
        btnCalcular = new javax.swing.JButton();
        lblResultado = new javax.swing.JLabel();

        setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
        setTitle("Caso 1 - Alquiler de vehículos");

        lblCliente.setText("Cliente:");
        pnlFormulario.add(lblCliente);

        cmbCliente.setModel(new javax.swing.DefaultComboBoxModel<>(new String[] { }));
        pnlFormulario.add(cmbCliente);

        lblDias.setText("Días:");
        pnlFormulario.add(lblDias);
```

```
        txtDias.setColumns(5);
        pnlFormulario.add(txtDias);

        lblPrecioBase.setText("Precio base por día:");
        pnlFormulario.add(lblPrecioBase);

        txtPrecioBase.setColumns(7);
        pnlFormulario.add(txtPrecioBase);

        btnCalcular.setText("Calcular");
        pnlFormulario.add(btnCalcular);

        lblResultado.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
        lblResultado.setText(" ");

        getContentPane().add(pnlFormulario, java.awt.BorderLayout.NORTH);
        getContentPane().add(lblResultado, java.awt.BorderLayout.CENTER);

        pack();
    }// </editor-fold>//GEN-END:initComponents

    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton btnCalcular;
    private javax.swing.JComboBox<String> cmbCliente;
    private javax.swing.JLabel lblCliente;
    private javax.swing.JLabel lblDias;
    private javax.swing.JLabel lblPrecioBase;
    private javax.swing.JLabel lblResultado;
    private javax.swing.JPanel pnlFormulario;
    private javax.swing.JTextField txtDias;
    private javax.swing.JTextField txtPrecioBase;
    // End of variables declaration//GEN-END:variables
}
```

## Paso 7. Controlador (ControladorAlquiler)

Llena el combo, escucha el botón y muestra el resultado.

### Archivo: ControladorAlquiler.java

```
package com.usil.controller;

import com.usil.model.TipoCliente;
import com.usil.service.ServicioAlquiler;
import com.usil.view.VistaAlquiler;
import java.util.Arrays;

public class ControladorAlquiler {

    private VistaAlquiler vista;
    private ServicioAlquiler servicio;

    public ControladorAlquiler(VistaAlquiler vista, ServicioAlquiler servicio) {
        this.vista = vista;
        this.servicio = servicio;

        // El combo se llena con los valores del enum usando streams.
        Arrays.stream(TipoCliente.values()).forEach(e ->
vista.getCmbCliente().addItem(e.name()));

        vista.getBtnCalcular().addActionListener(e -> calcular());
    }

    public void iniciar() {
        vista.setLocationRelativeTo(null);
        vista.setVisible(true);
    }
```

```
    private void calcular() {
        try {
            TipoCliente tipo =
TipoCliente.values()[vista.getCmbCliente().getSelectedIndex()];
            int dias = Integer.parseInt(vista.getTxtDias().getText().trim());
            double precioBase =
Double.parseDouble(vista.getTxtPrecioBase().getText().trim());
            if (dias <= 0 || precioBase <= 0) {
                vista.mostrarError("Los valores deben ser mayores que 0.");
                return;
            }
            double total = servicio.calcular(tipo, dias, precioBase);
            vista.getLblResultado().setText(String.format("Total a pagar: S/ %.2f", total));
        } catch (NumberFormatException ex) {
            vista.mostrarError("Ingrese números válidos.");
        }
    }
}
```

## Paso 8. Crear la clase Principal

Inicia el programa.

### Archivo: Principal.java

```
package com.usil;

import com.usil.controller.ControladorAlquiler;
import com.usil.service.ServicioAlquiler;
import com.usil.view.VistaAlquiler;
import javax.swing.SwingUtilities;
import javax.swing.UIManager;

public class Principal {

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            try {
                UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
            } catch (Exception ex) {
                // se usa el aspecto por defecto de Swing
            }
            new ControladorAlquiler(new VistaAlquiler(), new ServicioAlquiler()).iniciar();
        });
    }
}
```

## Paso 9. Ejecutar y probar

Clic derecho sobre Principal.java, Run File. Se ingresan los datos de prueba y se presiona el botón.

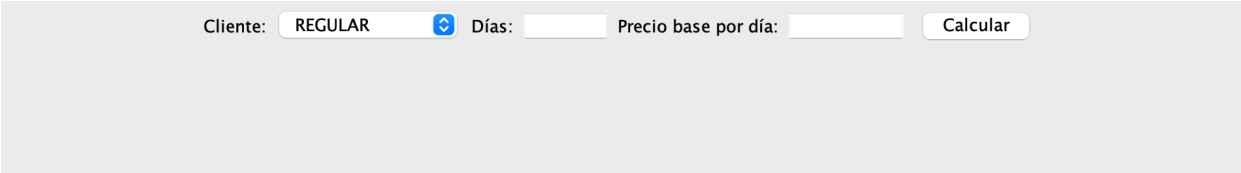



Figura 1. Ventana inicial.

| Cliente | Días | Precio base | Resultado |
|---|---|---|---|
| REGULAR | 7 | 100 | S/ 700.00 |
| CORPORATIVO | 7 | 100 | S/ 595.00 |
| PREMIUM | 7 | 100 | S/ 525.00 |

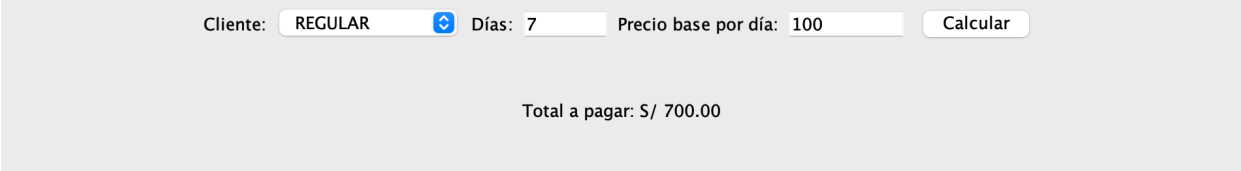



Figura 2. REGULAR: S/ 700.00.

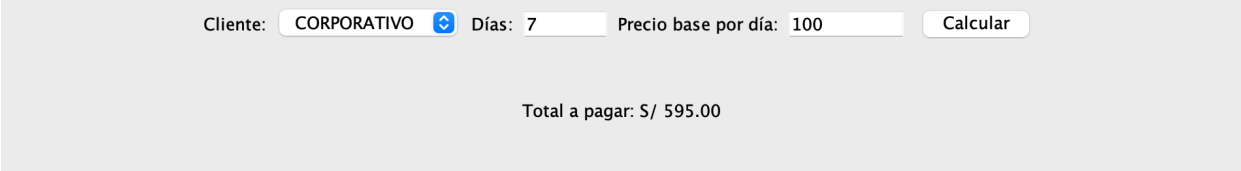



Figura 3. CORPORATIVO: S/ 595.00.

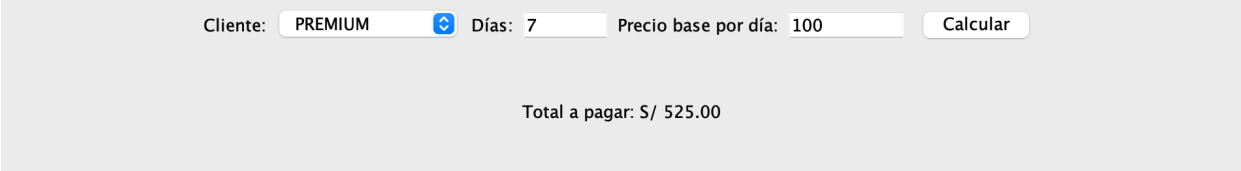



Figura 4. PREMIUM: S/ 525.00.